Servidor HTTP com Qualidade de Serviço em C e Pthreads: controle de taxa por IP, admissão e estimativa de RTT (Versão 2)

Rodrigo Silveira, João Gabriel Oliveira da Silva, Nicolas Ramos

Curso de Engenharia de Computação – Universidade Federal do Pampa (UNIPAMPA)
Campus Bagé – Bagé, RS – Brasil

Fundamentos e Avaliação de Redes de Computadores – 2026/2 – Relatório da segunda versão

Abstract. *This report presents the second version of an HTTP/1.1 server written in C with POSIX threads. Building on the first version (concurrent clients and persistent connections), it adds Quality of Service mechanisms: per-source-IP rate control for every object except HTML, with the rate of an address shared among its simultaneous connections through a per-IP virtual clock; admission control bounded by the server's maximum throughput (HTTP 503); and per-client estimates of round-trip time, end-to-end delay and bandwidth, obtained from an HTML request followed by the request for its first object and shown in a real-time panel. Experiments over loopback with distinct source addresses show measured rates within 0.7% of the configured values, a fair split among three connections of the same address, and correct admission decisions. We also show that Nagle's algorithm combined with delayed acknowledgments inflated the measured RTT from under 1 ms to about 41 ms, and how disabling it fixed the measurement.*

Resumo. *Este relatório apresenta a segunda versão de um servidor HTTP/1.1 escrito em C com Pthreads. Partindo da primeira versão (clientes concorrentes e conexões persistentes), ela acrescenta mecanismos de Qualidade de Serviço: controle de taxa por IP de origem para todos os objetos, exceto o HTML, com a taxa de um endereço dividida entre as suas conexões simultâneas por meio de um relógio virtual por IP; controle de admissão limitado pela vazão máxima do servidor (HTTP 503); e estimativas, por cliente, de RTT, atraso fim-a-fim e largura de banda, obtidas a partir da requisição do HTML seguida da requisição do seu primeiro objeto e exibidas em um painel em tempo real. Experimentos em loopback, com endereços de origem distintos, mostram taxas medidas a menos de 0,7% das configuradas, divisão justa entre três conexões do mesmo endereço e decisões de admissão corretas. Mostra-se também que o algoritmo de Nagle, combinado com o ACK atrasado, inflava o RTT medido de menos de 1 ms para cerca de 41 ms, e como desativá-lo corrigiu a medida.*

1. Introdução

Aplicações diferentes impõem requisitos distintos de qualidade de serviço (QoS) à rede: algumas toleram atraso, outras dependem de vazão garantida [Kurose e Ross 2013; Forouzan 2013]. Do lado do servidor, uma forma simples de oferecer tratamento diferenciado é controlar a taxa com que cada cliente é atendido e recusar novos clientes quando a capacidade contratada se esgota.

O trabalho prático da disciplina pede um servidor HTTP em linguagem C com Pthreads, desenvolvido em versões incrementais. A primeira versão, já entregue, atende vários clientes simultaneamente e segue o HTTP/1.1 com conexões persistentes [Fielding et al. 2022b]. Esta segunda versão acrescenta os mecanismos de QoS previstos na especificação: (i) envio dos objetos, exceto o arquivo HTML, com taxa controlada conforme o IP de origem, lida de um arquivo auxiliar, e 1000 kbps para IPs não cadastrados, se viável; (ii) divisão da taxa entre conexões simultâneas do mesmo IP; (iii) controle de admissão que não permita exceder a vazão máxima do servidor, informada como parâmetro; e (iv) apresentação em tempo real dos clientes atendidos, com estimativas de atraso fim-a-fim e largura de banda baseadas no RTT.

A Seção 2 descreve a arquitetura e os algoritmos, a Seção 3 apresenta a avaliação experimental e a Seção 4 conclui. O código está em https://github.com/nicolasURamos/t1-redes.

2. Desenvolvimento

2.1. Arquitetura

O servidor mantém o modelo da primeira versão: uma thread destacada por conexão, que trata todas as requisições daquela conexão até um `Connection: close`, um erro ou 15 s de inatividade. A versão 2 introduz dois módulos novos (Tabela 1) e liga-os ao código existente em poucos pontos. A Figura 1 mostra o caminho de uma conexão.

Tabela 1. Organização do código (módulos novos da versão 2 em destaque).

Arquivo	Responsabilidade
`servidor.c`	Parâmetros, *socket*, `accept`, *timeouts*, `TCP_NODELAY`; uma thread por conexão, que pede admissão ao iniciar
`http.c`	Leitura incremental, validação, laço *keep-alive*; registra o instante de cada requisição
`arquivos.c`	Caminho seguro, MIME, envio em blocos; aplica o ritmo do IP (exceto HTML)
`qos.c`	Tabela de taxas, tabela de IPs ativos, admissão, relógio virtual por IP, medidas de RTT
`painel.c`	Thread que redesenha o painel a cada 1 s

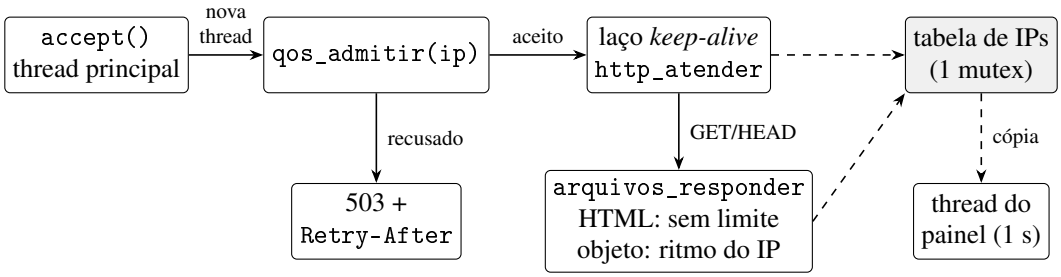


Figura 1. Caminho de uma conexão na versão 2. Setas tracejadas indicam acesso ao estado compartilhado.

A execução recebe quatro parâmetros opcionais: `./servidor [porta] [pasta] [vazao_max_kbps] [arquivo_taxas]`. O arquivo de taxas tem uma linha `IP taxa_kbps` por endereço, admite comentários com # e gera aviso para linhas inválidas. IPs ausentes recebem 1000 kbps.

2.2. Controle de taxa compartilhado por IP

Cada IP com conexões ativas possui um registro com a sua taxa R (kbps) e um relógio virtual T_{prox}, o instante a partir do qual o próximo bloco daquele IP pode ser enviado. Antes de enviar um bloco de n bytes de um objeto que não seja HTML, a conexão executa, com o mutex travado:

$$t_{\text{ini}} = \max\left(t_{\text{agora}}, T_{\text{prox}}\right), \qquad T_{\text{prox}} \leftarrow t_{\text{ini}} + \frac{8n}{1000\,R}\ [\text{s}] \tag{1}$$

Em seguida, destrava o mutex, dorme até t_{ini} com `clock_nanosleep` em modo absoluto sobre `CLOCK_MONOTONIC` e só então envia o bloco. Como todas as conexões do mesmo IP reservam intervalos no mesmo relógio, a soma das suas taxas nunca ultrapassa R e a divisão entre elas sai automática, inclusive quando conexões abrem ou fecham no meio de uma transferência. Essa abordagem dispensa calcular R/k para k conexões, o que exigiria reajustar todas as conexões a cada mudança de k. O termo $\max(\cdot)$ impede que um IP ocioso acumule crédito e depois envie uma rajada, de modo semelhante a um balde de fichas de profundidade mínima [Forouzan 2013].

O tamanho do bloco corresponde a cerca de 20 ms de dados na taxa do IP ($n = 1000R/8/50$ bytes), limitado entre 1460 e 65536 bytes. Blocos pequenos suavizam o envio em taxas baixas; o limite inferior evita chamadas de sistema excessivas. O HTML é enviado em blocos de 64 KiB e sem espera, como exige a especificação.

2.3. Controle de admissão

A admissão é decidida por endereço. Uma conexão de um IP que já tem conexões ativas é sempre aceita e passa a dividir a taxa daquele IP. Um IP novo, de taxa R, só é aceito se $C + R \leq V_{\max}$, onde C é a soma das taxas dos IPs ativos (vazão comprometida) e $V_{\max}$ é a vazão máxima do servidor. Ao fechar a última conexão de um IP, a sua taxa é subtraída de C. Assim, "1000 kbps se viável" significa que um IP não cadastrado só entra se houver 1000 kbps livres.

O cliente recusado recebe `503 Service Unavailable` com `Retry-After: 5` [Fielding et al. 2022a]. Fechar o *socket* logo após escrever o 503, com a requisição do cliente ainda não lida, faria o *kernel* enviar um RST, e o cliente poderia perder a resposta. Por isso o servidor fecha apenas o lado de escrita (`shutdown`) e lê o que restar por até 1 s antes de fechar. Para que essa espera nunca atrase o `accept`, a admissão é feita no início da thread de cada conexão.

2.4. Estimativa de RTT, atraso e largura de banda

Seguindo a especificação, a estimativa usa duas requisições consecutivas do mesmo cliente: a do HTML e a do primeiro objeto referenciado nele. Quando o servidor termina de enviar um HTML, registra o instante da requisição ($t_{\text{req,html}}$), o instante do fim do envio ($t_{\text{fim,html}}$) e o tamanho (B bytes). Na chegada da requisição seguinte de um objeto daquele IP ($t_{\text{req,obj}}$), calcula:

$$\text{RTT} = t_{\text{req,obj}} - t_{\text{fim,html}}, \qquad \text{atraso} \approx \frac{\text{RTT}}{2}, \qquad \text{banda} = \frac{8B}{t_{\text{req,obj}} - t_{\text{req,html}}} \tag{2}$$

O intervalo medido inclui o tempo que o cliente leva para processar o HTML; o atraso RTT/2 é, portanto, uma estimativa superior. A banda é a vazão efetiva de um ciclo completo, do pedido do HTML ao pedido do objeto; com HTML pequeno (694 bytes), ela é dominada pelo RTT e subestima a capacidade do enlace. Como navegadores podem atrasar ou paralelizar os pedidos de imagens, os experimentos usam `wget` e `curl`, como sugere a especificação. Cada medida também é registrada no log do servidor.

2.5. Painel em tempo real

Uma thread dedicada copia, a cada segundo, o estado da tabela de IPs (com o mutex travado apenas durante a cópia) e imprime o painel: vazão máxima, comprometida e livre; e, por IP, o número de conexões, a taxa configurada, a taxa real no último segundo, o RTT, o atraso e a banda. Em terminal, a tela é redesenhada; redirecionado para arquivo, os quadros se acumulam, o que permite registrar a evolução de um experimento.

2.6. Processo de desenvolvimento e verificação

O desenvolvimento seguiu etapas incrementais versionadas com Git: parâmetros e arquivo de taxas; admissão; controle de taxa; medidas e painel; documentação e *scripts* de experimento. Cada etapa só avançou com compilação sem avisos (`-Wall -Wextra -Wpedantic`) e com os testes automatizados passando. Aos seis testes de integração da versão 1 foram somados seis testes de QoS, que usam endereços de origem distintos no *loopback* (127.0.0.1, .2 e .3), já que o Linux trata todo o bloco 127.0.0.0/8 como local e o servidor enxerga cada endereço como um cliente diferente (Tabela 2). Uma execução dos testes com o ThreadSanitizer não apontou condições de corrida no acesso ao estado compartilhado.

Tabela 2. Testes automatizados de QoS (`make test`), com vazão máxima de 10000 kbps.

Teste	Critério de aprovação
HTML sem limite de taxa	$< 0{,}3$ s
1 download de 3,1 MB a 8000 kbps	entre 85% e 125% de 3,15 s
2 downloads simultâneos do mesmo IP	cada um entre 85% e 125% de 6,29 s
IP não cadastrado, 125 kB (1000 kbps)	entre 0,75 e 1,3 s
Admissão (8000 + 1000 ocupados, pedido de 4000)	503 com `Retry-After`; aceito após liberar
RTT no painel após HTML + objeto	valor numérico < 50 ms

Uso de inteligência artificial. Conforme exigido pela especificação, registra-se que o código e a documentação da versão 1 tiveram apoio do ChatGPT (OpenAI) e que a implementação da versão 2 (`qos.c`, `painel.c`, alterações em `servidor.c`, `http.c` e `arquivos.c`, testes de QoS e *script* de experimentos) teve apoio do Claude Code (Anthropic). O apoio está indicado no comentário de topo de cada arquivo, no README e no histórico do repositório. A equipe revisou, compilou e testou o código.

3. Avaliação Experimental

3.1. Ambiente e metodologia

Os experimentos foram executados em um computador com Windows e WSL2 (Ubuntu 22.04, GCC 11.4), com servidor e clientes na mesma máquina, pela interface de *loopback*. Para representar clientes distintos, o *script* `scripts/experimentos.sh` usa `curl --interface` com os endereços de origem 127.0.0.1, 127.0.0.2 e 127.0.0.3. O servidor foi iniciado com vazão máxima de 12000 kbps e com o arquivo de taxas do repositório (127.0.0.1: 8000 kbps; 127.0.0.2: 4000 kbps; 127.0.0.3: 2000 kbps). O objeto transferido é `imagem1.bmp`, de 3.145.782 bytes (25,17 Mbit). O tempo esperado de uma transferência é o tamanho em bits dividido pela taxa da conexão, e a taxa medida é calculada a partir da velocidade média informada pelo `curl` (bytes/s $\times 8/1000$).

3.2. Resultados

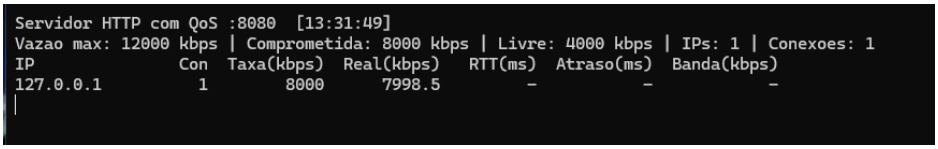
A Tabela 3 resume os cenários. Em todos, a taxa medida ficou a menos de 0,7% da taxa prevista. No cenário com três downloads simultâneos do mesmo IP, cada conexão recebeu cerca de um terço dos 8000 kbps e a soma (8021 kbps) ficou dentro do limite do endereço, o que confirma a divisão da taxa. No cenário com dois IPs, cada endereço recebeu a sua própria taxa ao mesmo tempo, sem interferência mútua. O desvio positivo, da ordem de 0,15%, vem do primeiro bloco de cada transferência, enviado sem espera, já que o relógio do IP ainda está no passado.

Tabela 3. Resultados dos experimentos (vazão máxima de 12000 kbps; objeto de 25,17 Mbit). O desvio é da taxa medida; na última linha, do tempo total.

Cenário	Taxa prevista (kbps)	Tempo previsto (s)	Tempo medido (s)	Taxa medida (kbps)	Desvio
1 download, 127.0.0.1	8000	3,146	3,141	8012	+0,15%
3 simultâneos, 127.0.0.1 (conexão 1)	2667	9,437	9,426	2670	+0,12%
3 simultâneos, 127.0.0.1 (conexão 2)	2667	9,437	9,381	2683	+0,61%
3 simultâneos, 127.0.0.1 (conexão 3)	2667	9,437	9,431	2668	+0,05%
2 IPs juntos: 127.0.0.1	8000	3,146	3,141	8012	+0,15%
2 IPs juntos: 127.0.0.2	4000	6,292	6,281	4007	+0,17%
`wget -p` (HTML + 3 imagens)	8000	9,437	9,46	–	+0,24%

Admissão. Com 127.0.0.1 (8000 kbps) e 127.0.0.3 (2000 kbps) baixando, a vazão comprometida era de 10000 kbps. Um pedido de 127.0.0.2, de 4000 kbps, exigiria 14000 kbps e recebeu 503, como registrado pelo cliente e pelo log do servidor (“Recusado (vazao maxima atingida): 127.0.0.2”). No cenário anterior, 127.0.0.1 e 127.0.0.2 juntos (12000 kbps) haviam sido aceitos, exatamente no limite.

Painel e medidas. A Figura 2 mostra o painel durante os experimentos: a taxa real de cada IP ficou em 7998,5 e 3999,4 kbps, para taxas configuradas de 8000 e 4000 kbps. O `wget -p` produziu a medida “RTT 0,988 ms | atraso 0,494 ms | banda 2161,2 kbps” no log. Valores abaixo de 1 ms são esperados no *loopback*, em que não há enlace físico, e são dominados pelo tempo de processamento do cliente; o próprio teste automatizado obteve 0,17 ms. Pelo mesmo motivo, a banda estimada reflete a duração do ciclo HTML → objeto, e não uma capacidade de enlace.



(a) Download de 127.0.0.1 (taxa configurada de 8000 kbps).



(b) Download de 127.0.0.2 (taxa configurada de 4000 kbps).

Figura 2. Painel do servidor durante os experimentos.

3.3. Efeito do algoritmo de Nagle na medida de RTT

Durante o desenvolvimento, a primeira medida de RTT no *loopback* foi de 41,8 ms, valor incompatível com uma comunicação local. A causa é a interação entre o algoritmo de Nagle [Nagle 1984] e o ACK atrasado [Braden 1989]: o servidor envia o cabeçalho e o

corpo da resposta em duas chamadas `send`; com Nagle ativo, o corpo, pequeno, fica retido até chegar o ACK do cabeçalho, e o cliente atrasa esse ACK por até cerca de 40 ms. Com `TCP_NODELAY` na conexão aceita, a mesma medida caiu para 0,74 ms. A mudança também reduz o tempo de entrega de respostas pequenas e não afeta os objetos grandes, que já são enviados em blocos completos.

3.4. Limitações

O *loopback* não impõe as restrições de um enlace real: os experimentos verificam a precisão do controle de taxa e da admissão implementados, mas não a variação de RTT e de banda em uma rede física. A comparação entre clientes em rede cabeada (IEEE 802.3) e sem fio (IEEE 802.11), com análise de capturas no Wireshark e monitoramento no IPTraf, está prevista para a validação com o professor. No WSL2, isso exige o modo de rede espelhado, pois a configuração padrão coloca o Linux atrás de NAT e faria todos os clientes externos aparecerem com o mesmo endereço.

4. Conclusões

A segunda versão atende aos requisitos de QoS da especificação. O relógio virtual por IP mostrou-se uma forma simples e precisa de limitar a taxa e de dividi-la entre conexões do mesmo endereço, com desvios abaixo de 0,7% nos experimentos. O controle de admissão aceitou clientes até o limite exato da vazão máxima e recusou o excedente com 503. O painel exibe em tempo real os clientes, as taxas configurada e real e as estimativas de RTT, atraso e banda. A investigação do RTT inflado por Nagle e ACK atrasado mostrou na prática como decisões da camada de transporte afetam medidas feitas na camada de aplicação. Como próximos passos, ficam os experimentos com clientes remotos em rede cabeada e sem fio, com capturas no Wireshark e IPTraf, e a avaliação com navegadores.

Referências

Anthropic (2026). *Claude Code*. Ferramenta de apoio à programação utilizada na versão 2. https://claude.ai/.

Braden, R. (1989). *Requirements for Internet Hosts – Communication Layers*. RFC 1122, IETF.

Comer, D. E. (2016). *Redes de computadores e Internet*. 6. ed. Porto Alegre: Bookman.

Fielding, R., Nottingham, M. e Reschke, J. (2022a). *HTTP Semantics*. RFC 9110, IETF.

Fielding, R., Nottingham, M. e Reschke, J. (2022b). *HTTP/1.1*. RFC 9112, IETF.

Forouzan, B. A. (2013). *Redes de computadores: uma abordagem top-down*. Porto Alegre: AMGH.

Kurose, J. F. e Ross, K. W. (2013). *Redes de computadores e a Internet: uma abordagem top-down*. 6. ed. São Paulo: Pearson.

Linux man-pages project (2026). *pthreads(7), clock_nanosleep(2), tcp(7)*. https://man7.org/linux/man-pages/.

Nagle, J. (1984). *Congestion Control in IP/TCP Internetworks*. RFC 896, IETF.

OpenAI (2026). *ChatGPT*. Ferramenta de apoio utilizada na versão 1. https://chatgpt.com/.

UNIPAMPA (2026). *Trabalho de Programação em Rede: especificação*. Fundamentos e Avaliação de Redes de Computadores, 2026/2.